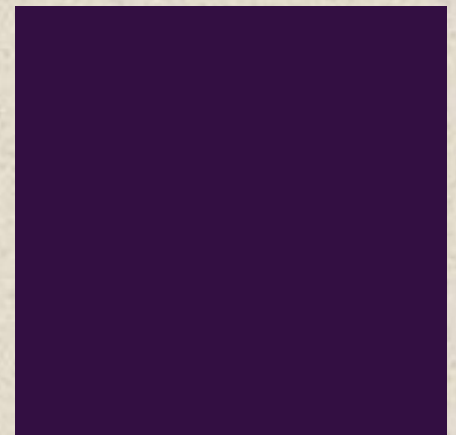
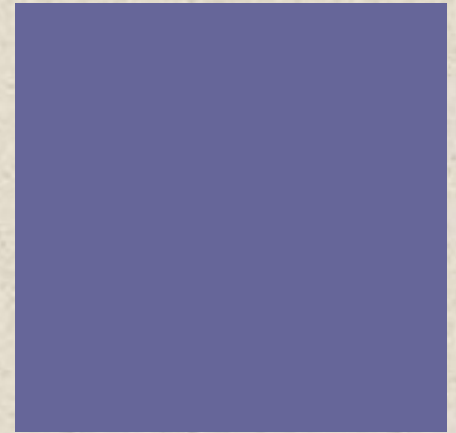
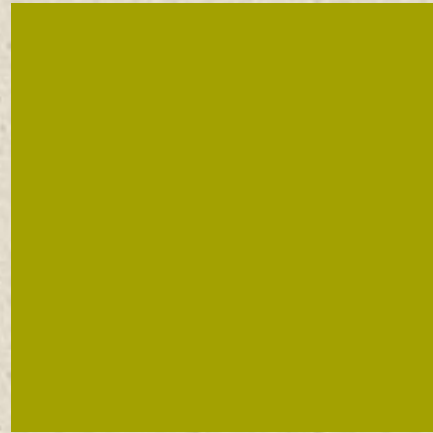




William Stallings Computer Organization and Architecture 10th Edition

© 2016 Pearson Education, Inc., Hoboken,
NJ. All rights reserved.



+ Chapter 2

Performance Issues

+ Designing for Performance

- The cost of computer systems continues to drop dramatically, while the performance and capacity of those systems continue to rise equally dramatically
- Today's laptops have the computing power of an IBM mainframe from 10 or 15 years ago
- Processors are so inexpensive that we now have microprocessors we throw away
- Desktop applications that require the great power of today's microprocessor-based systems include:
 - Image processing
 - Three-dimensional rendering
 - Speech recognition
 - Videoconferencing
 - Multimedia authoring
 - Voice and video annotation of files
 - Simulation modeling



+ Designing for Performance

- Businesses rely on increasingly powerful servers to handle transaction and database processing and to support massive client/server networks that have replaced the huge mainframe computer centers of yesteryear
- Cloud service providers use massive high-performance banks of servers to satisfy high-volume, high-transaction-rate applications for a broad spectrum of clients



+ Designing for Performance

- As the number of transistors and clock speed increase, chip performance increases correspondingly
- Other techniques are also used to increase performance
- Computing power is only useful when it is kept busy with a smooth flow of useful work
- This is why it is often necessary to look beyond clock speed when comparing processors.



+ Microprocessor Speed

Techniques built into contemporary processors include:

- **Pipelining** – Processor moves data or instructions into a conceptual pipe with all stages of the pipe processing simultaneously
 - This works like an assembly line where the next instruction begins execution before the previous instruction is completed
- **Branch prediction** – Processor looks ahead in the instruction code fetched from memory and predicts which branches, or groups of instructions, are likely to be processed next
 - “Guess” what is likely to be needed next and load it into a cache before it is actually needed
- **Superscalar execution** – This is the ability to issue more than one instruction in every processor clock cycle.
 - In effect, multiple parallel pipelines are used.

+ Microprocessor Speed

Techniques built into contemporary processors include:

- **Data flow analysis** – Processor analyzes which instructions are dependent on each other's results, or data, to create an optimized schedule of instructions
 - Instructions may be performed in a different order than specified in the program as long as the result is the same
- **Speculative execution** – Using branch prediction and data flow analysis, some processors speculatively execute instructions ahead of their actual appearance in the program execution, holding the results in temporary locations, keeping execution engines as busy as possible
 - “Guess” what will be required next and start doing it
 - If the guess is wrong, throw away the unnecessary work

+ Performance Balance

- One difficulty in designing an efficient system is that different components operate at different speeds
 - For example DRAM is generally much slower than the processor
- It is necessary to adjust the organization and architecture to compensate for this mismatch
- Overall balance in the system is more important than the raw performance of any one component
- This is why computer benchmarks are used to compare system performance

+ Performance Balance

- To overcome the imbalance between memory and processor speeds there are several approaches
 - Increase the number of bits that are retrieved at one time by making DRAMs “wider” rather than “deeper” and by using wide bus data paths – 8, 16, 32, 64 bit systems
 - Reduce the frequency of memory access by incorporating increasingly complex and efficient cache structures between the processor and main memory
 - Change the DRAM interface to make it more efficient by including a cache or other buffering scheme on the DRAM chip
 - Increase the interconnect bandwidth between processors and memory by using higher speed buses and a hierarchy of buses to buffer and structure data flow

+ I/O Devices

- Another area of design focus is the handling of I/O devices.
- Some of these devices create tremendous data throughput demands
- The processor can usually handle this throughput, but there is the problem of getting the data from the processor to the device
- Strategies here include:
 - Caching and buffering schemes
 - The use of higher-speed interconnection buses and more elaborate structures of buses
 - The use of multiple-processor configurations can aid in satisfying I/O demands

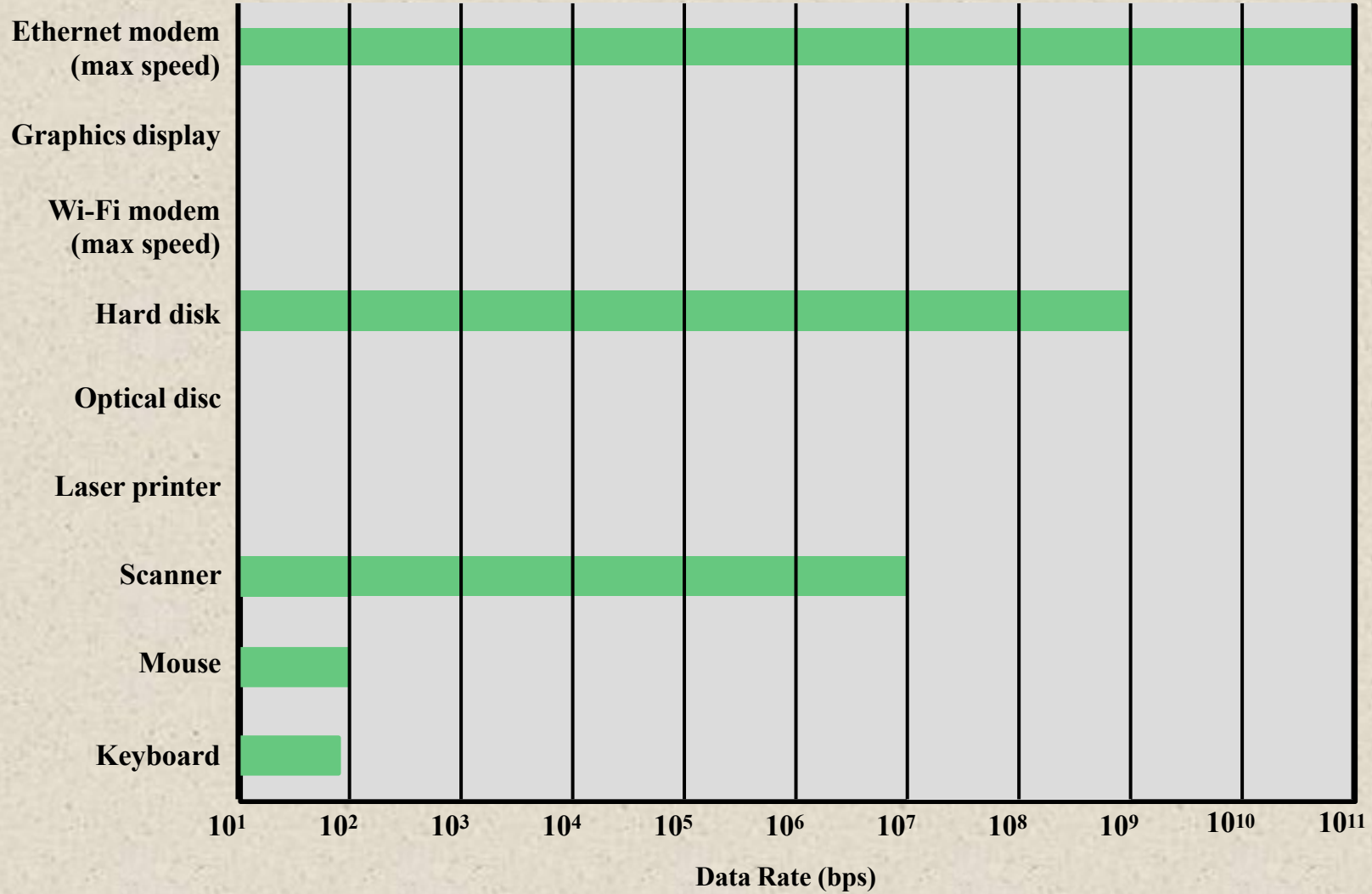
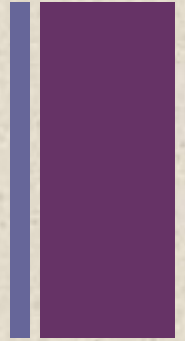
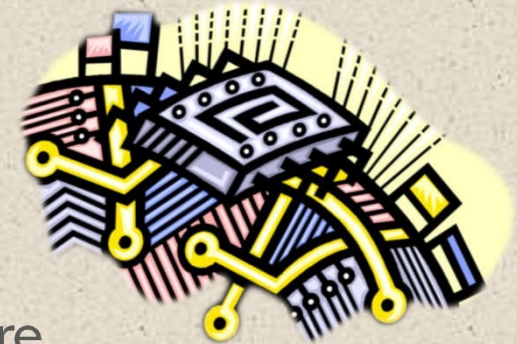


Figure 2.1 Typical I/O Device Data Rates

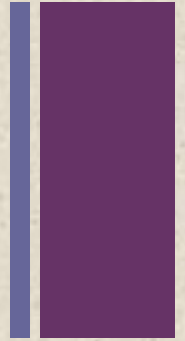
+ Improvements in Chip Organization and Architecture



- Increase hardware speed of processor
 - Fundamentally due to shrinking logic gate size
 - More gates, packed more tightly, increasing clock rate
 - Propagation time for signals reduced
- Increase size and speed of caches
 - Dedicated part of processor chip
 - Cache access times drop significantly
- Change processor organization and architecture
 - Increase effective speed of instruction execution
 - Parallelism



+ Problems with Clock Speed and Logic Density



■ Power

- Power density increases with density of logic and clock speed
- Dissipating heat

■ RC delay

- Speed at which electrons flow limited by resistance and capacitance of metal wires connecting them
- Delay increases as the RC product increases
- As components on the chip decrease in size, the wire interconnects become thinner, increasing resistance
- Also, the wires are closer together, increasing capacitance

■ Memory latency

- Memory speeds lag processor speeds

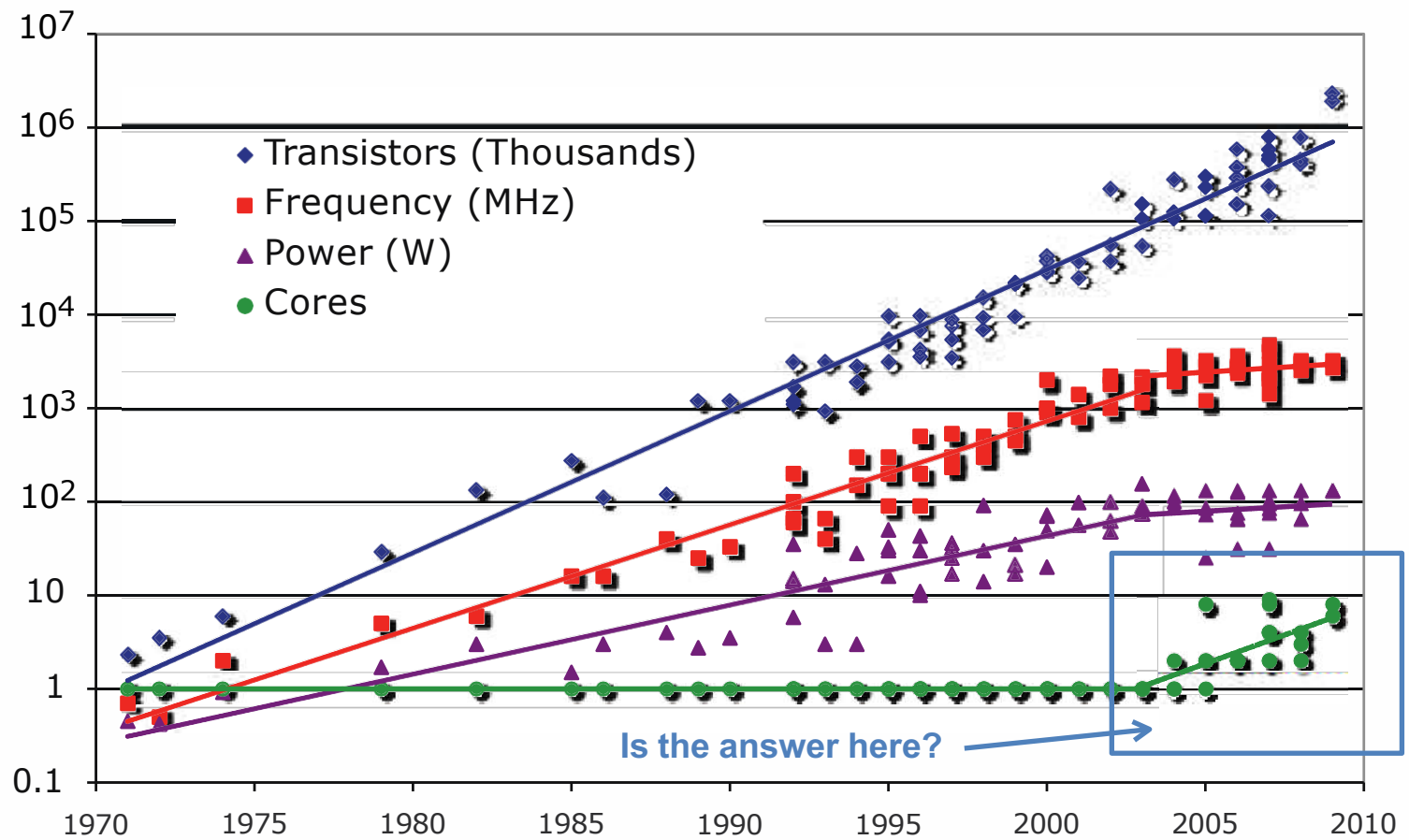
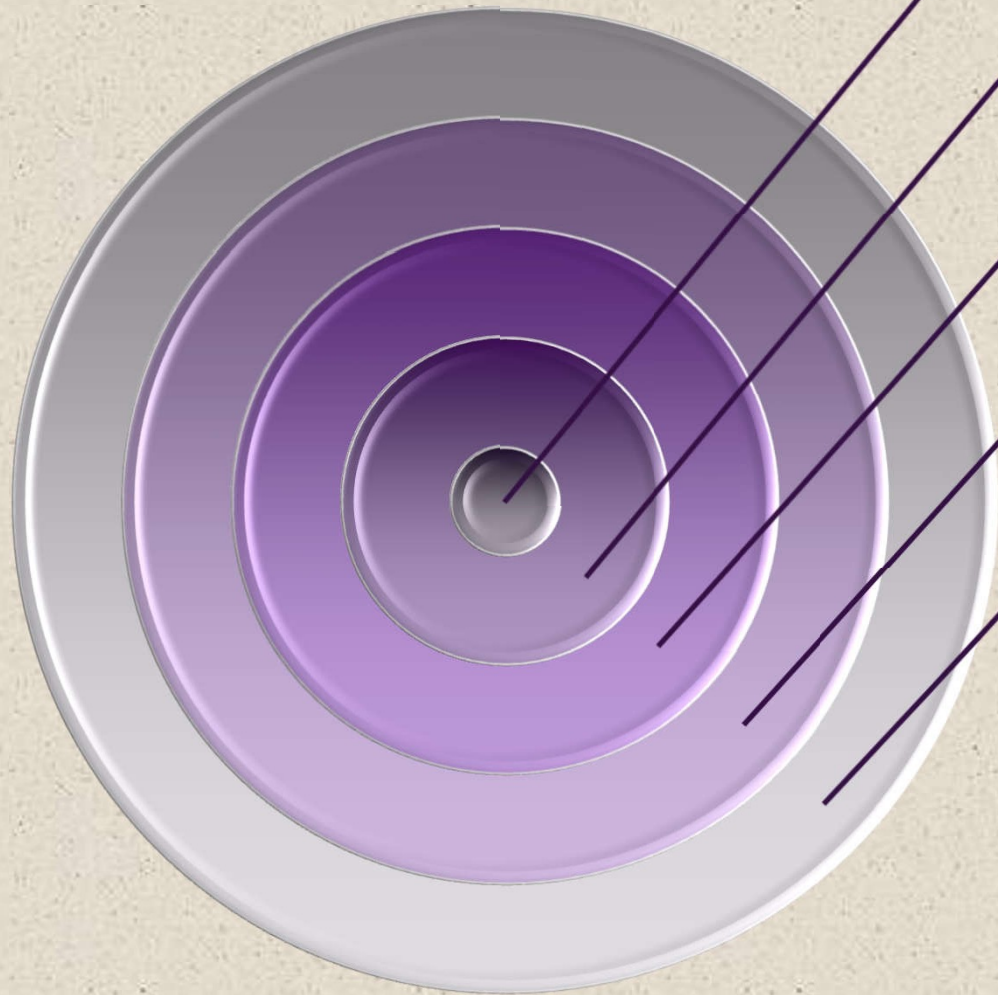


Figure 2.2 Processor Trends

Multicore



same clock speed

The use of multiple processors on the same chip provides the potential to increase performance without increasing the clock rate

simpler element

Strategy is to use two simpler processors on the chip rather than one more complex processor

more cache

With two processors larger caches are justified

and more complex cache

As caches become larger it makes performance sense to create two and then three levels of cache on a chip

shorter wire

Most signals remain within a single core making RC delay is less of an issue

+ Many Integrated Core (MIC) Graphics Processing Unit (GPU)

MIC

- Leap in performance as well as the challenges in developing software to exploit such a large number of cores
- The multicore and MIC strategy involves a homogeneous collection of general purpose processors on a single chip

The most recent trend is to include **Heterogeneous** cores

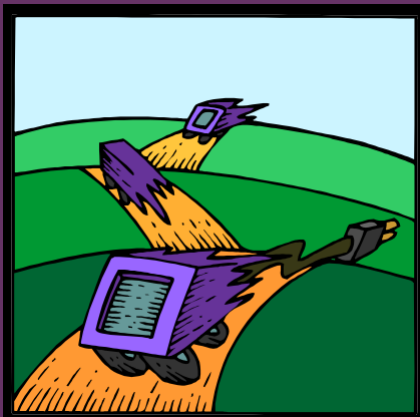
GPU

- Core designed to perform parallel operations on graphics data
- Traditionally found on a plug-in graphics card, it is used to encode and render 2D and 3D graphics as well as process video
- Used as vector processors for a variety of applications that require repetitive computations

Widely used for Machine Learning especially Deep Learning



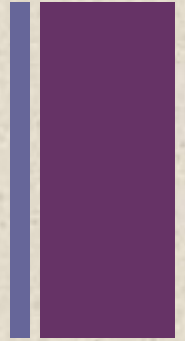
Amdahl's Law



- Gene Amdahl
- Deals with the potential speedup of a program using multiple processors compared to a single processor
- Diminishing returns as processors spend more time communicating with each other and less time actually solving the problem
- Illustrates the problems facing industry in the development of multi-core machines

Can / must be tackled by designing effective interconnection or communication protocol

+ Amdahl's Law



- Software usually needs to be rewritten to take advantage of multiple cores
- Some parts of the program may not be able to take advantage of multiple cores at all
 - Let f be the proportion of the program that can take advantage of multiple cores and N be the number of cores

$$\text{speedup} = \frac{1}{(1-f) + \frac{f}{N}}$$

Insight?

Not everything can be
parallelized

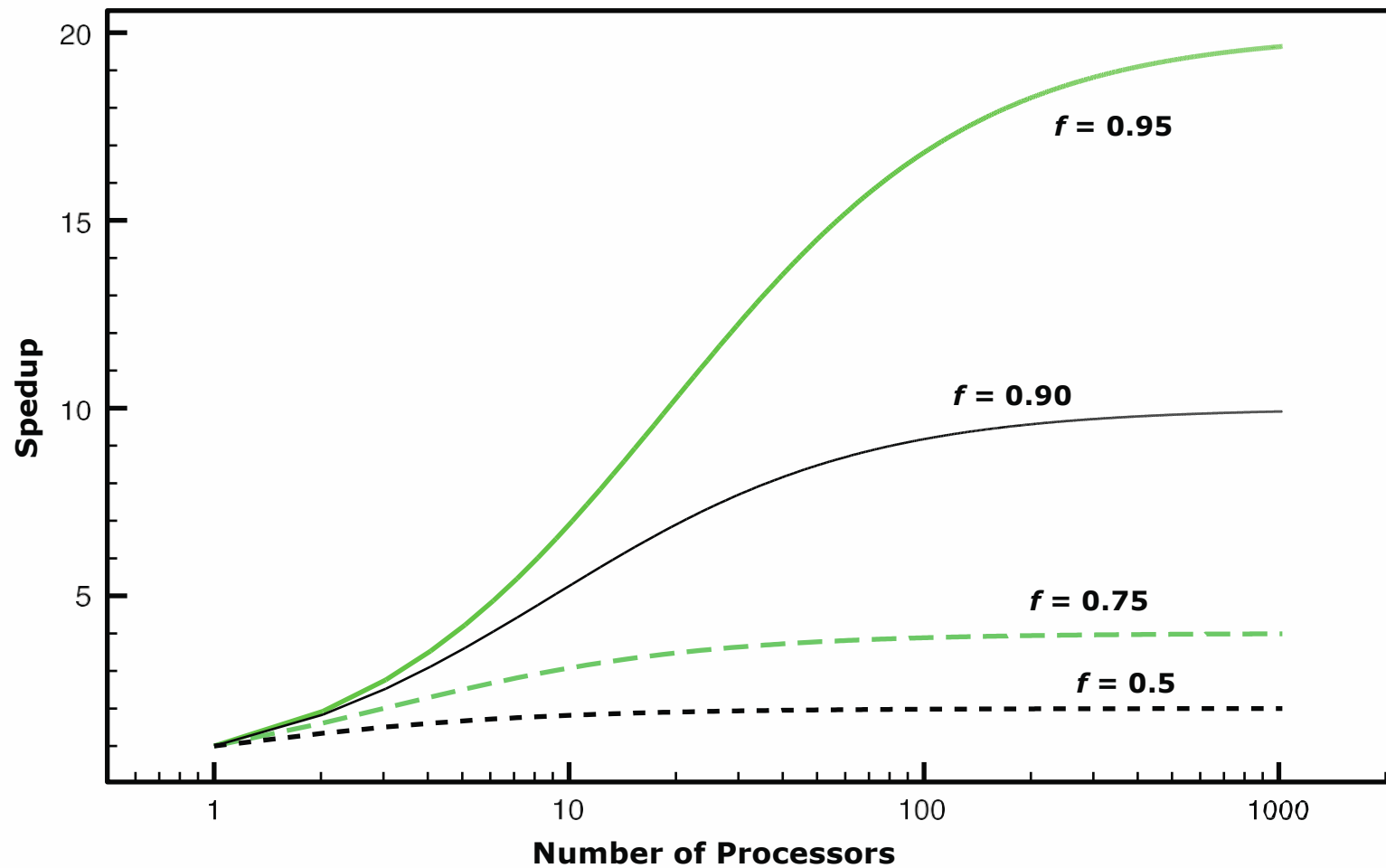
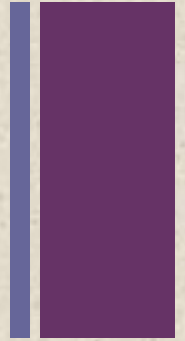


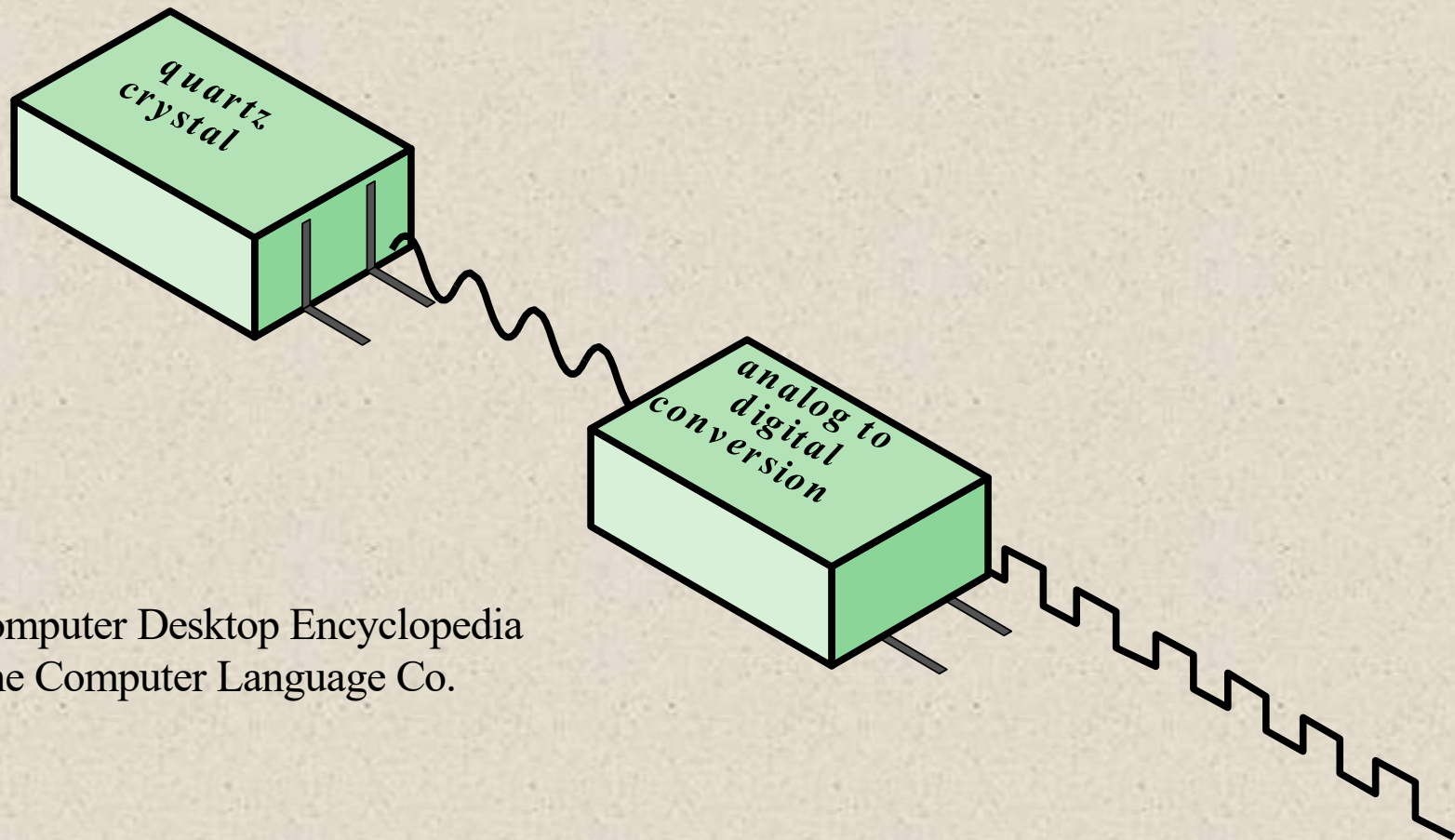
Figure 2.4 Amdahl's Law for Multiprocessors

+ Clock Speed



- The speed of a processor is dictated by the pulse frequency produced by a system clock
 - A quartz crystal which generates a constant sine wave while power is applied
 - The wave is converted into a digital signal
- Clock speed is measured in cycles per second (Hertz)
 - A 1-GHz processor received 1 billion pulses per second
- If the frequency is f then the cycle time is $\tau = 1 / f$

Period and frequency



From Computer Desktop Encyclopedia
1998, The Computer Language Co.

Figure 2.5 System Clock

+ Cycles per Instruction

- The processor will have many different instructions it can perform and each will take a fixed number of cycles
- The average number of cycles per instruction is the CPI.
 - RISC processors generally have a low CPI while CISC processors have a higher CPI
- CPI is determined by three factors
 - p – the average number of cycles to decode and execute the instruction
 - m – the average number of memory references needed
 - k – the ratio between memory cycle time and processor cycle time

+ Time to Execute a Program

- Different architectures will take a different number of instructions to execute a given program
 - RISC processors generally take more instructions than CISC processors
- If the number of instructions to complete a program is I_c then the time to execute the program is given by

$$T = I_c \times [p + (m \times k)] \times \tau$$

- These parameters are influenced by four system attributes



	I_c	p	m	k	τ
Instruction set architecture	X	X			
Compiler technology	X	X	X		
Processor implementation		X			X
Cache and memory hierarchy				X	X

Table 2.1 Performance Factors and System Attributes

+ MIPS Rate and MFLOPS

- A common measure of performance is the rate at which instructions are executed
- MIPS – millions of instructions per second

$$\text{MIPS rate} = \frac{I_c}{T \times 10^6} = \frac{f}{CPI \times 10^6}$$

But are these
representative
metrics?

- Another common performance measure is the number of floating point operations per second MFLOPS
 - This is often used in scientific computing where much of the work is manipulating floating point numbers

+ MIPS Rate and MFLOPS

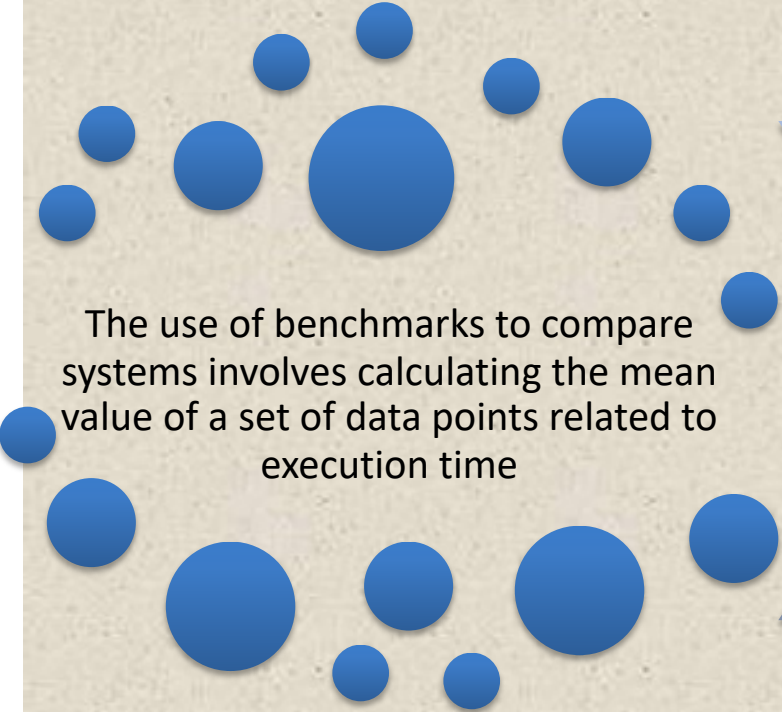
EXAMPLE 2.2 Consider the execution of a program that results in the execution of 2 million instructions on a 400-MHz processor. The program consists of four major types of instructions. The instruction mix and the *CPI* for each instruction type are given below, based on the result of a program trace experiment:

Instruction Type	<i>CPI</i>	Instruction Mix (%)
Arithmetic and logic	1	60
Load/store with cache hit	2	18
Branch	4	12
Memory reference with cache miss	8	10

} Weighted
average
(CPI)

The average *CPI* when the program is executed on a uniprocessor with the above trace results is $CPI = 0.6 + (2 \times 0.18) + (4 \times 0.12) + (8 \times 0.1) = 2.24$. The corresponding MIPS rate is $(400 \times 10^6)/(2.24 \times 10^6) \approx 178$.

Calculating the Mean



The use of benchmarks to compare systems involves calculating the mean value of a set of data points related to execution time



The three common formulas used for calculating a mean are:

- Arithmetic
- Geometric
- Harmonic

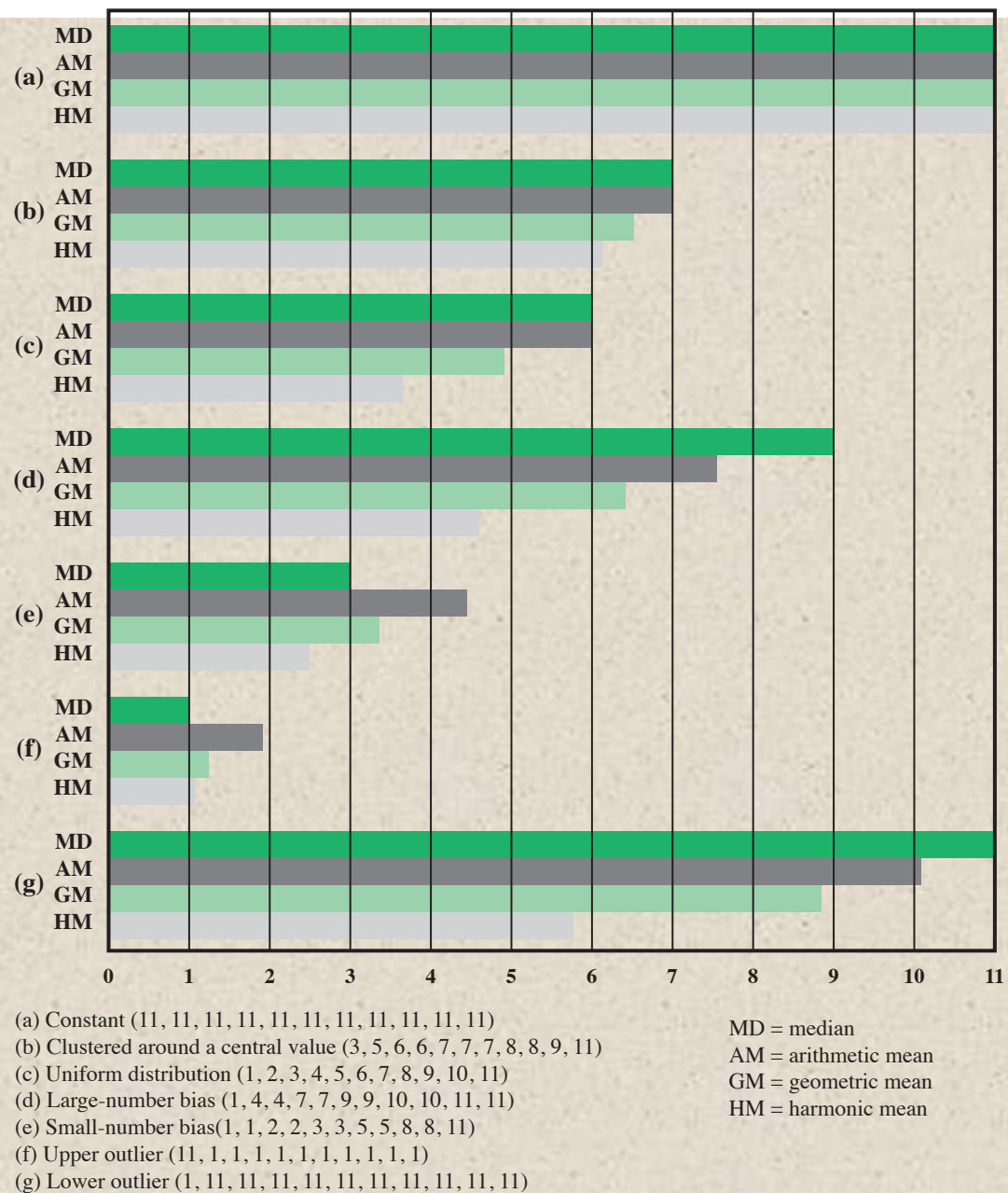
Calculating the Mean

If a and b are positive numbers, then

$$\text{Arithmetic Mean (AM)} = \frac{a + b}{2}$$

$$\text{Geometric Mean (GM)} = \sqrt{ab}$$

$$\text{Harmonic Mean (HM)} = \frac{2ab}{a + b} = \frac{(GM)^2}{AM}$$



**Figure 2.6 Comparison of Means on Various Data Sets
(each set has a maximum data point value of 11)**

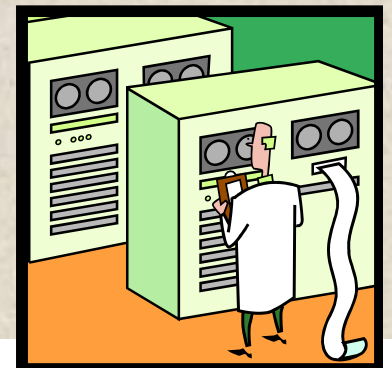
- An Arithmetic Mean (AM) is an appropriate measure if the sum of all the measurements is a meaningful and interesting value
- The AM is a good candidate for comparing the execution time performance of several systems

Arithmetic

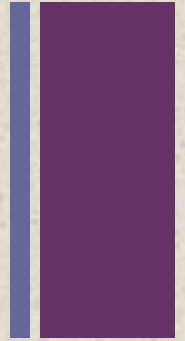
Mean

For example, suppose we were interested in using a system for large-scale simulation studies and wanted to evaluate several alternative products. On each system we could run the simulation multiple times with different input values for each run, and then take the average execution time across all runs. The use of multiple runs with different inputs should ensure that the results are not heavily biased by some unusual feature of a given input set. The AM of all the runs is a good measure of the system's performance on simulations, and a good number to use for system comparison.

- + ■ The AM used for a time-based variable, such as program execution time, has the important property that it is directly proportional to the total time
 - If the total time doubles, the mean value doubles

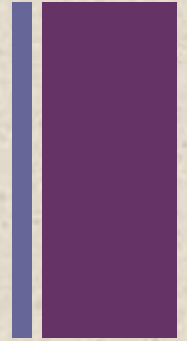


+ Benchmarks



- There are many statistics that can influence a system's performance
- What users care about is the overall performance running a particular type of program
- Benchmarks are used to summarize the performance of a system running examples of actual code

+ Benchmark Principles

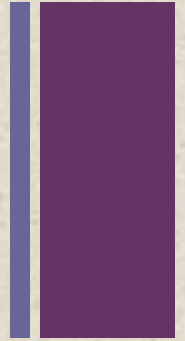


■ Desirable characteristics of a benchmark program:

1. It is written in a high-level language, making it portable across different machines
2. It is representative of a particular kind of programming domain or paradigm, such as systems programming, numerical programming, or commercial programming
3. It can be measured easily
4. It has wide distribution



+ System Performance Evaluation Corporation (SPEC)



■ Benchmark suite

- A collection of programs, defined in a high-level language
- Together attempt to provide a representative test of a computer in a particular application or system programming area

■ SPEC

- An industry consortium
- Defines and maintains the best known collection of benchmark suites aimed at evaluating computer systems
- Performance measurements are widely used for comparison and research purposes



SPEC CPU2006



- Best known SPEC benchmark suite
- Industry standard suite for processor intensive applications
- Appropriate for measuring performance for applications that spend most of their time doing computation rather than I/O 
- Consists of 17 floating point programs written in C, C++, and Fortran and 12 integer programs written in C and C++
- Suite contains over 3 million lines of code
- Fifth generation of processor intensive suites from SPEC

Benchmark	Reference time (hours)	Instr count (billion)	Language	Application Area	Brief Description
400.perlbench	2.71	2,378	C	Programming Language	PERL programming language interpreter, applied to a set of three programs.
401.bzip2	2.68	2,472	C	Compression	General-purpose data compression with most work done in memory, rather than doing I/O.
403.gcc	2.24	1,064	C	C Compiler	Based on gcc Version 3.2, generates code for Opteron.
429.mcf	2.53	327	C	Combinatorial Optimization	Vehicle scheduling algorithm.
445.gobmk	2.91	1,603	C	Artificial Intelligence	Plays the game of Go, a simply described but deeply complex game.
456.hmmcr	2.59	3,363	C	Search Gene Sequence	Protein sequence analysis using profile hidden Markov models.
458.sjeng	3.36	2,383	C	Artificial Intelligence	A highly ranked chess program that also plays several chess variants.
462.libquantum	5.76	3,555	C	Physics / Quantum Computing	Simulates a quantum computer, running Shor's polynomial-time factorization algorithm.
464.h264ref	6.15	3,731	C	Video Compression	H.264/AVC (Advanced Video Coding) Video compression.
471.omnetpp	1.74	687	C++	Discrete Event Simulation	Uses the OMNet++ discrete event simulator to model a large Ethernet campus network.
473.astar	1.95	1,200	C++	Path-finding Algorithms	Pathfinding library for 2D maps.
483.xalanbmk	1.92	1,184	C++	XML Processing	A modified version of Xalan-C++, which transforms XML documents to other document types.

Table 2.5

SPEC CPU2006 Integer Benchmarks

(Table can be found on page 69 in the textbook.)

Benchmark	Reference time (hours)	Instr count (billion)	Language	Application Area	Brief Description
410.bwaves	3.78	1,176	Fortran	Fluid Dynamics	Computes 3D transonic transient laminar viscous flow.
416.gamess	5.44	5,189	Fortran	Quantum Chemistry	Quantum chemical computations.
433.milc	2.55	937	C	Physics / Quantum Chromodynamics	Simulates behavior of quarks and gluons
434.zeusmp	2.53	1,566	Fortran	Physics / CFD	Computational fluid dynamics simulation of astrophysical phenomena.
435.gromacs	1.98	1,958	C, Fortran	Biochemistry / Molecular Dynamics	Simulate Newtonian equations of motion for hundreds to millions of particles.
436.cactusADM	3.32	1,376	C, Fortran	Physics / General Relativity	Solves the Einstein evolution equations.
437.leslie3d	2.61	1,273	Fortran	Fluid Dynamics	Model fuel injection flows.
444.namd	2.23	2,483	C++	Biology / Molecular Dynamics	Simulates large biomolecular systems.
447.dealII	3.18	2,323	C++	Finite Element Analysis	Program library targeted at adaptive finite elements and error estimation.
450.soplex	2.32	703	C++	Linear Programming, Optimization	Test cases include railroad planning and military airlift models.
453.povray	1.48	940	C++	Image Ray-tracing	3D Image rendering.
454.calculix	2.29	3,041	C, Fortran	Structural Mechanics	Finite element code for linear and nonlinear 3D structural applications.
459.GemsFDTD	2.95	1,320	Fortran	Computational Electromagnetics	Solves the Maxwell equations in 3D.
465.tonto	2.73	2,392	Fortran	Quantum Chemistry	Quantum chemistry package, adapted for crystallographic tasks.
470.lbm	3.82	1,500	C	Fluid Dynamics	Simulates incompressible fluids in 3D.
481.wrf	3.10	1,684	C, Fortran	Weather	Weather forecasting model
482.sphinx3	5.41	2,472	C	Speech recognition	Speech recognition software.

Table 2.6

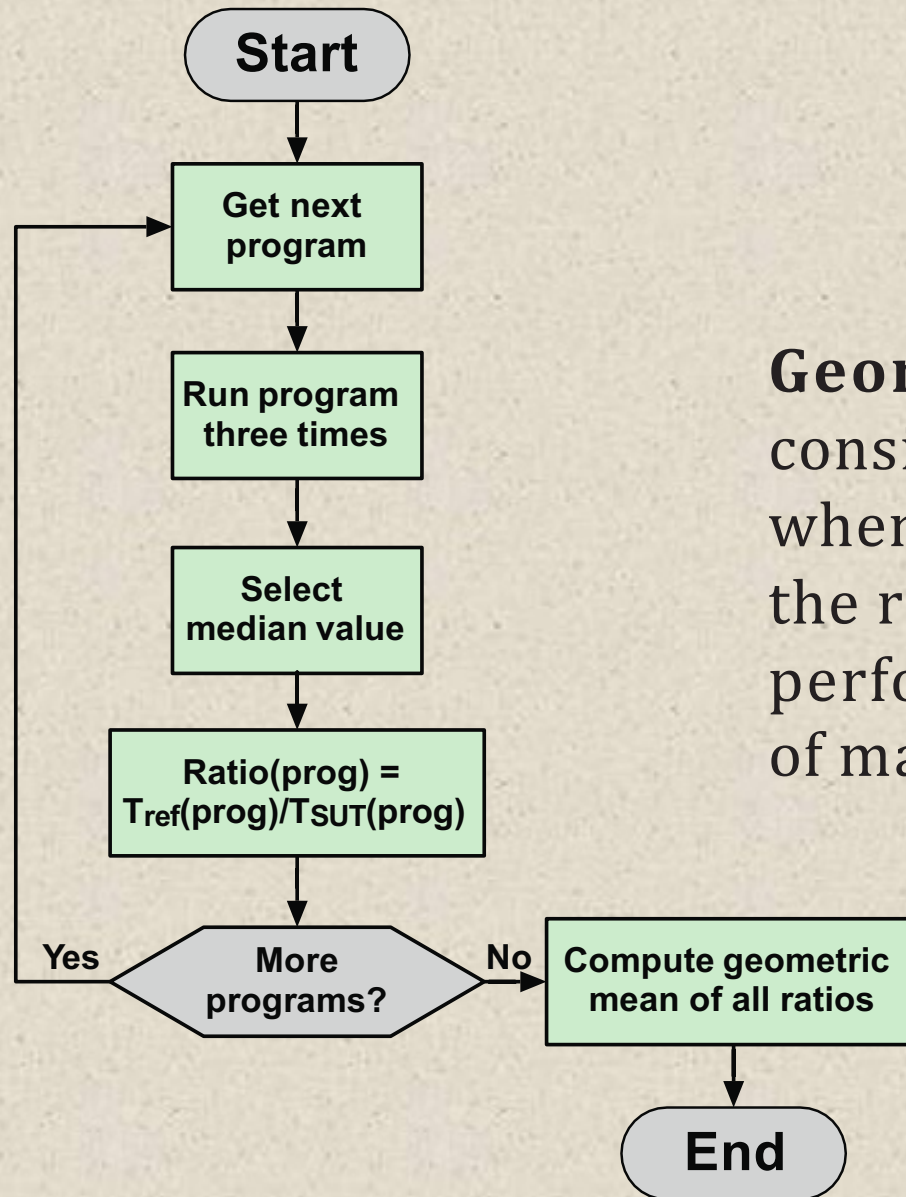
SPEC

CPU2006

Floating-Point

Benchmarks

(Table can be found on page 70 in the textbook.)



Geometric mean
consistent results
when measuring
the relative
performance
of machines

Figure 2.7 SPEC Evaluation Flowchart

Table 2.7 Some SPEC CINT2006 Results

(a) Sun Blade 1000

Benchmark	Execution time	Execution time	Execution time	Reference time	Ratio
400.perlbench	3077	3076	3080	9770	3.18
401.bzip2	3260	3263	3260	9650	2.96
403.gcc	2711	2701	2702	8050	2.98
429.mcf	2356	2331	2301	9120	3.91
445.gobmk	3319	3310	3308	10490	3.17
456.hmmer	2586	2587	2601	9330	3.61
458.sjeng	3452	3449	3449	12100	3.51
462.libquantum	10318	10319	10273	20720	2.01
464.h264ref	5246	5290	5259	22130	4.21
471.omnetpp	2565	2572	2582	6250	2.43
473.astar	2522	2554	2565	7020	2.75
483.xalancbmk	2014	2018	2018	6900	3.42

(b) Sun Blade X6250

Benchmark	Execution time	Execution time	Execution time	Reference time	Ratio	Rate
400.perlbench	497	497	497	9770	19.66	78.63
401.bzip2	613	614	613	9650	15.74	62.97
403.gcc	529	529	529	8050	15.22	60.87
429.mcf	472	472	473	9120	19.32	77.29
445.gobmk	637	637	637	10490	16.47	65.87
456.hmmer	446	446	446	9330	20.92	83.68
458.sjeng	631	632	630	12100	19.18	76.70
462.libquantum	614	614	614	20720	33.75	134.98
464.h264ref	830	830	830	22130	26.66	106.65
471.omnetpp	619	620	619	6250	10.10	40.39
473.astar	580	580	580	7020	12.10	48.41
483.xalancbmk	422	422	422	6900	16.35	65.40

+ Summary

Chapter 2

Performance Issues

- Designing for performance
 - Microprocessor speed
 - Performance balance
 - Improvements in chip organization and architecture
 - Multicore
 - MICs
 - GPUs
 - Amdahl's Law
- Basic measures of computer performance
 - Clock speed
 - Instruction execution rate
 - Calculating the mean
 - Benchmark principles
 - SPEC benchmarks